

Semester Project Proposal On:

AGENTIC RAG WITH SELF-CORRECTION AND CITATION HIGHLIGHTING

Submitted to:

Kathmandu University

School of Engineering

Department of Artificial Intelligence

Dhulikhel-3, Kavrepalanchowk, Nepal

Date: May 8, 2026

Semester Project Proposal

Course Code: AISP 222

Submitted in partial fulfillment of the requirements for
2nd Year, 4th Semester, B.Tech AI (Batch 2024)

Submitted by:

Darshan Karna (Roll No. 22739) | Batch 2024

Aayush Thapa (Roll No. 22751) | Batch 2024

Ghanshyam Ghimire (Roll No. 22673) | Batch 2024

Submission Date: May 8, 2026

Abstract

Large Language Models (LLMs) have transformed information retrieval, yet their tendency to hallucinate factually unsupported content remains a fundamental barrier to deployment in high-stakes domains. Retrieval-Augmented Generation (RAG) mitigates this by grounding generation in retrieved documents; however, standard single-pass RAG pipelines remain vulnerable to faithfulness failures when the model selectively ignores context or blends it with parametric knowledge (Lewis et al., 2020). This project proposes an Agentic RAG architecture that introduces a multi-agent self-correction loop to systematically detect and eliminate hallucinations before output. A primary Generation Agent (Llama-3-8B-Instruct) drafts responses from retrieved context, while a Verification Agent (`cross-encoder/nli-deberta-base`) evaluates each output sentence via Natural Language Inference (NLI), classifying it as entailment, neutral, or contradiction against source chunks. Flagged sentences are iteratively revised through a LangGraph-orchestrated correction loop until all claims are grounded. The verified entailment mappings additionally power sentence-level, interactive citations in a React.js and Tailwind CSS frontend. Evaluation using the RAGAS framework targets a faithfulness score above 0.90 and a Correction Delta above 0.15. This work advances trustworthy, transparent AI for document-grounded question answering without requiring LLM fine-tuning.

Keywords: Retrieval-Augmented Generation, Hallucination, Natural Language Inference, Agentic Workflow, Self-Correction, Citation, LangGraph, DeBERTa, Faithfulness

Contents

Abstract	i
Abbreviations	vi
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope and Limitations	3
1.5 Motivation	3
2 Related Works	5
2.1 Self-Reflective Retrieval-Augmented Generation	5
2.2 Self-Correcting Agentic Graph RAG for Clinical Decision Support	5
2.3 Sub-Sentence Citations for Retrieval-Augmented Generation	6
2.4 Agentic Orchestration and Evaluation Frameworks	6
3 Methodology	8
3.1 Document Ingestion and Chunking	8
3.2 Semantic Retrieval	8
3.3 Generation Agent	8
3.4 Verification Agent	9
3.5 Self-Correction Loop	9
3.6 Citation UI Pipeline	10
3.7 Evaluation Protocol	10
4 Tools and Technologies	12
4.1 Software Specifications	13
4.1.1 Core AI Libraries	13
4.1.2 Backend and API	13

4.1.3	Frontend Tools	13
4.1.4	Development Environment	13
4.2	Hardware Specifications	14
4.2.1	Minimum Requirements	14
4.2.2	Recommended Requirements	14
5	Project Planning	15
6	Expected Outcomes	17
7	Ethical Considerations	18
7.1	Data Privacy	18
7.2	Bias and Fairness	18
7.3	Transparency and Accountability	18
8	Risk Analysis	20
9	Conclusion	21

List of Figures

Figure 3.1	System Architecture of the Agentic RAG Pipeline	11
Figure 5.1	Project Timeline (15-Week Gantt Chart)	15

List of Tables

Table 4.1	Technology Stack	12
Table 6.1	Evaluation Metrics and Targets	17
Table 8.1	Risk Register	20

Abbreviations

AI	– Artificial Intelligence
API	– Application Programming Interface
ASGI	– Asynchronous Server Gateway Interface
CSS	– Cascading Style Sheets
DPR	– Dense Passage Retrieval
FAISS	– Facebook AI Similarity Search
GPU	– Graphics Processing Unit
JSON	– JavaScript Object Notation
LLM	– Large Language Model
NLI	– Natural Language Inference
NLP	– Natural Language Processing
PII	– Personally Identifiable Information
QA	– Question Answering
RAG	– Retrieval-Augmented Generation
RAGAS	– Retrieval-Augmented Generation Assessment
REST	– Representational State Transfer
UI	– User Interface
VRAM	– Video Random Access Memory

Chapter 1

Introduction

The rapid proliferation of Large Language Models (LLMs) has fundamentally altered how information is accessed, synthesised, and communicated. Systems capable of answering complex questions in natural language are now deployed across healthcare, legal research, customer support, and education. Yet a persistent and well-documented failure mode undermines their trustworthiness: hallucination—the generation of linguistically fluent but factually unsupported content [8]. As LLMs scale, the problem does not disappear; it becomes harder to detect because the generated text reads with increasing confidence and coherence even when it is wrong.

Retrieval-Augmented Generation (RAG) was introduced as a principled response to this challenge by conditioning generation on externally retrieved documents, thereby grounding the model in verifiable facts rather than parametric memory alone [1]. In practice, however, standard single-pass RAG pipelines still exhibit significant faithfulness failures. The model may selectively ignore retrieved passages, blend retrieved facts with conflicting pre-trained knowledge, or fabricate plausible-sounding but unsupported citations [14]. These failures are especially dangerous in domains where accuracy is paramount.

This project proposes a solution that goes beyond the single-pass paradigm: an Agentic RAG system that treats generation as an iterative, multi-agent process. By introducing a dedicated Natural Language Inference (NLI) verification step and a programmatic self-correction loop, the system can identify and eliminate hallucinations before the response reaches the user. The verified entailment mappings are further used to provide sentence-level, interactive citations in the user interface—turning a black-box system into a transparent and auditable one.

1.1 Background

The concept of augmenting language models with external retrieval was formalised by Lewis et al. [1], who demonstrated significant improvements in knowledge-intensive NLP tasks by combining dense retrieval with sequence-to-sequence generation. Subsequent work has explored adaptive retrieval [13], iterative retrieval [15], and self-reflective generation [2].

Despite these advances, faithfulness, defined as the property that every claim in the generated output is logically supported by the retrieved context remains an unsolved problem. Studies using the RAGAS evaluation framework consistently report faithfulness scores well below 1.0 even for state-of-the-art RAG systems [5]. The gap is particularly pronounced for long-form answers where the model must synthesise information across multiple retrieved chunks.

Concurrently, the field of Natural Language Inference has matured significantly. Models such as DeBERTa [7] trained on large NLI corpora can determine whether a hypothesis is entailed by, contradicted by, or neutral with respect to a given premise. Applying NLI as a post-hoc verification step for generated text has shown promise in earlier hallucination detection research [6]. This project operationalises that approach within a full agentic pipeline orchestrated by LangGraph.

Agentic frameworks represent the most recent evolution of LLM-based systems, enabling models to act as autonomous decision-makers within structured workflows [16]. LangGraph, the graph-based extension of LangChain [19], provides the necessary primitives for implementing stateful, cyclic agent workflows, a critical requirement for a self-correcting pipeline that must loop until all verification conditions are met.

1.2 Problem Statement

Despite the widespread adoption of RAG as the de facto grounding method for LLMs, production pipelines lack a built-in faithfulness guarantee. Standard systems follow a linear generate-once paradigm with no mechanism to verify whether each generated claim is actually supported by the retrieved passages. The “lost in the middle” phenomenon further compounds this: LLMs tend to prioritise information at the edges of long contexts, disregarding relevant content in the middle [10]. Additionally, existing systems provide no transparent attribution mechanism; users have no way to trace a generated claim back to its source document, making it impossible to audit the system’s reasoning or catch errors independently.

This project directly addresses these gaps by proposing an architectural solution: an agentic pipeline that internalises verification and correction as first-class operations, and surfaces the verification results as an interactive, sentence-level citation interface for the end user.

1.3 Objectives

1. To design and implement a multi-agent Agentic RAG pipeline that separates generation, verification, and correction into discrete, composable agents orchestrated by LangGraph.
2. To integrate a Natural Language Inference model (`cross-encoder/nli-deberta-base`) as a sentence-level fact-checker that evaluates every generated claim against retrieved source chunks.
3. To implement a programmatic self-correction loop that iteratively revises flagged sentences until all output claims are grounded in the retrieved context, with a bounded maximum iteration count.
4. To develop a React.js frontend with interactive, sentence-level citation highlighting that maps every verified claim to its source document chunk, providing full transparency to the user.

1.4 Scope and Limitations

The scope of this project is bounded to a document-grounded question answering system operating over a curated set of technical PDFs. The system supports single-turn queries; multi-turn conversational context is outside the current scope. The self-correction loop is limited to a maximum of three iterations per query to maintain acceptable response latency. The system does not support real-time web retrieval; all knowledge is sourced from the pre-indexed document corpus.

A key limitation is that the NLI verifier operates at the sentence level and may struggle with claims that require multi-hop reasoning across multiple chunks. Additionally, the faithfulness of the system is only as good as the quality of the retrieved chunks: if retrieval fails to surface the relevant passage, the verifier will correctly flag the generated claim, but the self-correction loop may also fail to produce a grounded rewrite. Domain adaptation for highly specialised vocabularies may require fine-tuning the NLI model, which is listed as an optional advanced step.

1.5 Motivation

The motivation for this project arises from a practical gap in the current landscape of deployed RAG systems: the absence of an automated, in-loop faithfulness guarantee. Every

hallucinated claim in a RAG system is invisible to the end user unless they manually cross-reference each sentence against the source document, a process that defeats the purpose of an AI assistant. By making verification and citation a native feature of the pipeline rather than a post-hoc analysis, this project aims to build a system that is not only accurate but also auditable and trustworthy.

From a research perspective, the project contributes a novel evaluation metric "Correction Delta" that directly quantifies the improvement attributable to the self-correction loop, enabling systematic benchmarking across different agent configurations. This fills a gap in existing evaluation frameworks, which measure final output quality but not the impact of intermediate correction steps.

Chapter 2

Related Works

The systems closest to what this project builds fall into three clusters: self-reflective RAG methods that teach the generator to judge its own output, agentic self-correction frameworks that attach that judgment to a graph-based loop, and fine-grained citation schemes that push attribution below the sentence level. Each cluster solves part of the problem but leaves something on the table.

2.1 Self-Reflective Retrieval-Augmented Generation

Asai et al. [2] rethought how language models use external knowledge. Rather than retrieving a fixed number of documents for every query regardless of context, Self-RAG lets the model decide on the fly whether retrieval is necessary at all. The mechanism relies on special “reflection tokens” inserted during generation: the model uses them to check whether its output is accurate, well-supported, and grounded in relevant passages. When retrieval does occur, the model scores each passage for relevance instead of treating all retrieved content as equally useful. This reduces the influence of misleading context and improves attribution. On open-domain QA, reasoning, and fact-verification benchmarks, Self-RAG outperformed ChatGPT and retrieval-augmented Llama2-chat, with the largest gains on long-form responses.

That said, Self-RAG has a structural problem this project cannot work around: the model is judge and defendant simultaneously. Because the reflection tokens come from the same weights that produced the answer, any systematic blind spot in the generator carries straight through into the verification step. The current project sidesteps this by using a separately trained NLI model, which means the two agents can genuinely disagree and that disagreement is where hallucinations get caught.

2.2 Self-Correcting Agentic Graph RAG for Clinical Decision Support

Hu et al. [3] tackled a harder version of the faithfulness problem: clinical question answering, where errors carry real consequences. Their system is built on a clinically verified hepatology knowledge graph and follows a state-driven “retrieve–evaluate–refine” loop in

which agents dynamically generate, validate, and optimize graph-based search strategies before assembling the final context. The iterative structure means weak or irrelevant evidence gets filtered at each step rather than silently passed through.

On a custom clinical QA dataset, the framework outperformed GPT-4, standard RAG, and Graph RAG on every measured metric, faithfulness (0.94), context recall (0.92), and answer relevancy (0.91). These numbers are worth noting not because this project targets hepatology, but because they show what a properly closed correction loop achieves in a demanding domain. The gap between those results and typical single-pass RAG scores is the practical motivation for building the self-correction architecture here.

2.3 Sub-Sentence Citations for Retrieval-Augmented Generation

A 2025 study on sub-sentence citations for RAG [4] identified a problem that sentence-level attribution misses: when a citation covers an entire sentence, it often bundles irrelevant material with the actual supporting evidence, and sometimes omits the specific phrase needed to verify the claim. Users end up reading more context than necessary, or not enough.

The fix proposed is to anchor citations at the phrase or span level, linking specific parts of the generated response directly to the portions of retrieved text that support them. The authors built annotation guidelines and a manually annotated dataset, and showed that finer-grained citation reduced user verification effort while maintaining completeness. For this project, the insight is that citation granularity matters beyond presentation. The current implementation operates at sentence level as a practical baseline, with sub-sentence attribution identified as a concrete direction for future work.

2.4 Agentic Orchestration and Evaluation Frameworks

Yao et al. [18] introduced ReAct, which interleaves chain-of-thought reasoning with tool-use actions in a single LLM, the foundational pattern for agentic pipelines. This project uses LangGraph [19], the graph-based extension of LangChain, which adds proper state management and conditional edges. The cyclic structure makes the self-correction loop practical: execution returns to the Generation Agent as many times as needed without unwinding the full pipeline.

For evaluation, Es et al. [5] built RAGAS, a reference-free framework measuring Faithfulness, Answer Relevance, and Context Recall for RAG systems. Faithfulness scores for state-of-the-art pipelines are consistently below 1.0, confirming the gap this project targets. The

Correction Delta metric introduced here, the faithfulness gain between the initial draft and the corrected output, xtends the RAGAS logic to measure what the loop itself contributes, not just the quality of the final answer.

Chapter 3

Methodology

The system implements a multi-stage RAG pipeline in which document retrieval, generation, verification, and correction are handled by specialized modules. Each stage draws on state-of-the-art techniques informed by prior research. The overall workflow is orchestrated as an agentic loop via LangGraph, iteratively refining output until every claim is grounded in the source documents [19]. Figure 3.1 provides an overview.

3.1 Document Ingestion and Chunking

Unstructured PDFs are processed with PyMuPDF to extract text. Cleaning removes headers, footers, and hyphenation artefacts. The text is then split into overlapping semantic chunks—500 tokens with a 50-token overlap—to preserve sentence continuity across boundaries. Each chunk is encoded using `sentence-transformers/all-MiniLM-L6-v2` [12], a Sentence-BERT model that produces dense embeddings comparable via cosine similarity with minimal compute. Chunk embeddings and metadata (document ID, page number, chunk index) are stored in a ChromaDB vector index backed by FAISS for fast retrieval.

3.2 Semantic Retrieval

At query time the user’s input is encoded with the same embedding model. A cosine similarity search over the indexed chunk embeddings returns the top- k relevant passages ($k = 5$ by default). Dense passage retrieval substantially outperforms classical BM25 on open-domain QA [9]. The retrieved chunks—with their IDs and metadata—form the context passed to the Generation Agent and retained as evidence for the Verification Agent.

3.3 Generation Agent

A Llama-3-8B-Instruct model served via Ollama locally, or via the Groq API for quantized inference, acts as the primary answer generator. The agent receives a structured prompt containing the user query and all retrieved context chunks. The system prompt instructs the model to “Answer strictly based on the provided context. Do not introduce any information not

in the context passages.” Output is written as standalone sentences, each stating one claim. This mirrors the RAG setup of Lewis et al. [1], which concatenates retrieved passages to the prompt so generation is grounded in external text. The model produces a fluent draft; it is not assumed to be fully faithful without the verification step.

During correction iterations, the prompt is augmented with the list of previously flagged sentences and their NLI verdicts, instructing the model to rewrite only those sentences.

3.4 Verification Agent

Each sentence in the generated draft is independently checked against the retrieved context using `cross-encoder/nli-deberta-base`, a DeBERTa-based cross-encoder fine-tuned on MNLI and related entailment data. DeBERTa achieves state-of-the-art results on NLI benchmarks [7]. For each sentence (hypothesis), entailment scores are computed against each retrieved chunk (premise). If the maximum entailment score across all chunks is ≥ 0.80 , the sentence is marked verified and mapped to the chunk yielding the highest score. Otherwise it is flagged. Thresholds in the 0.70–0.80 range are standard in NLI-based fact-checking [6]. The agent outputs a report of the form (*sentence*, *verdict*, *supporting_chunk_id*). Sentences marked Entailment are accepted; those marked Neutral or Contradiction are flagged for correction.

3.5 Self-Correction Loop

The generator and verifier are embedded in a LangGraph agent workflow with a conditional loop. After the first draft is scored, the workflow checks: are any sentences flagged as unsupported? If yes, the workflow re-invokes the Generation Agent with an augmented prompt that highlights only the flagged sentences—for example: “The following statement was not supported by the retrieved context. Rewrite it using only the retrieved context facts.” The model revises only those sentences. Up to three total iterations are allowed. If a sentence still cannot be grounded after three attempts, the system omits it or replaces it with a transparency disclaimer. This fail-safe prevents infinite loops and enforces honesty about the limits of the retrieved evidence [2, 3].

3.6 Citation UI Pipeline

Once all sentences pass verification, the final answer is assembled as a JSON payload mapping each sentence to its supporting chunk ID. The React.js and Tailwind CSS frontend renders each sentence as a `<span>` that highlights on hover or click, showing the source chunk text and document metadata. Any unverified sentence—if one remains—appears in a warning style. This provides interactive, sentence-level citations so users can audit every claim against its source. The design is directly motivated by the sub-sentence citation work of [4], which demonstrated that fine-grained attribution meaningfully reduces the effort required for verification.

3.7 Evaluation Protocol

The system is evaluated on held-out QA tasks using the RAGAS framework [5]. Three metrics are measured: Faithfulness (fraction of final-answer sentences entailed by retrieved context), Answer Relevance (semantic similarity of the answer to the question), and Context Recall (coverage of ground-truth evidence in retrieved chunks). A fourth metric—Correction Delta—is introduced here: the faithfulness improvement between the uncorrected first draft and the final corrected output. The single-pass RAG baseline is expected to produce hallucinated sentences; after correction, the target is Faithfulness > 0.90 and Correction Delta > 0.15 , representing at least a 15-point gain from the loop. This framing draws on the atomic-claim scoring approach of Min et al. [11] and directly quantifies what the agentic loop contributes to factual accuracy.

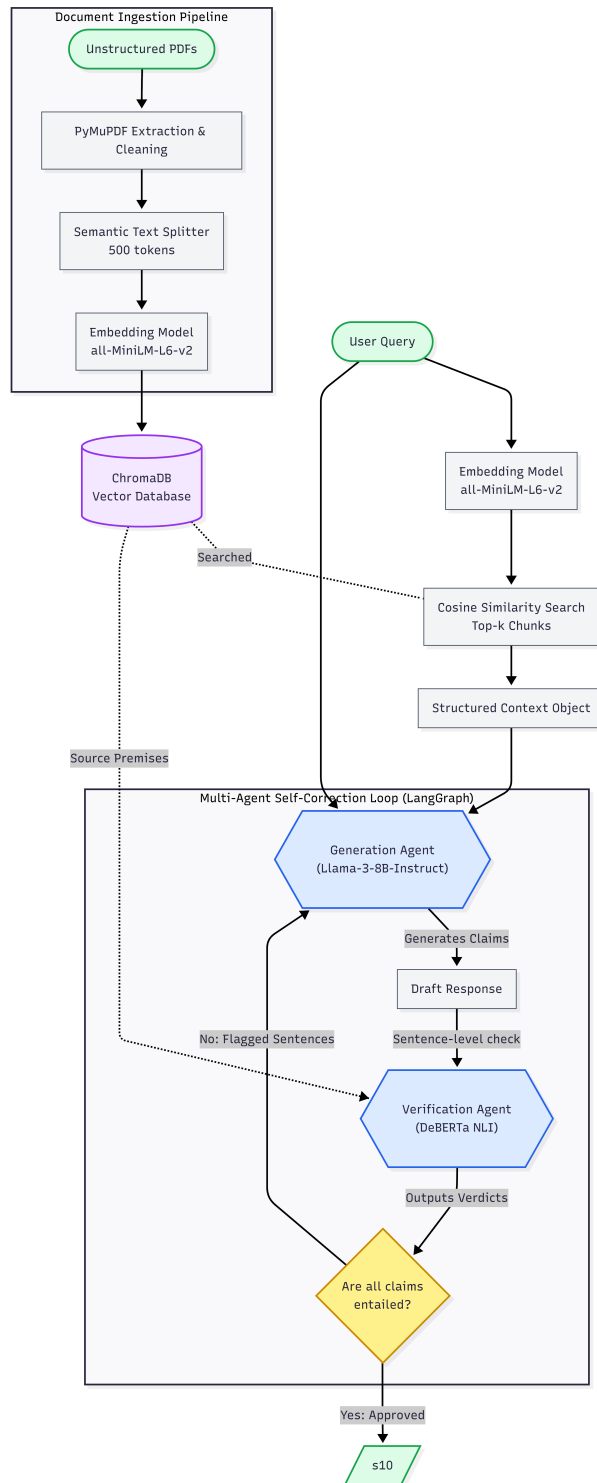


Figure 3.1: System Architecture of the Agentic RAG Pipeline

Chapter 4

Tools and Technologies

Table 4.1 summarises the full technology stack. Detailed specifications follow.

Table 4.1: Technology Stack

Component	Technology	Purpose
Generation Agent	Llama-3-8B-Instruct (Ollama / Groq)	Drafts answers from retrieved context
Embedding Model	all-MiniLM-L6-v2	Dense vector encoding of documents & queries
Verification Agent	cross-encoder/ nli-deberta-base	Sentence-level entailment classification
Vector Store	ChromaDB / FAISS	Embedding storage and cosine similarity search
Orchestration	LangGraph	Cyclic agentic workflow state machine
Backend	Python + FastAPI	REST API server
Frontend	React.js + Tailwind CSS	Citation-highlighting chat UI
Evaluation	RAGAS Framework	Faithfulness, Answer Relevance, Correction Delta

4.1 Software Specifications

4.1.1 Core AI Libraries

- `sentence-transformers` $\geq 2.5.0$ — Document and query embedding (`all-MiniLM-L6-v2`)
- `transformers` $\geq 4.40.0$ (HuggingFace) — DeBERTa NLI model loading and inference
- `torch` $\geq 2.2.0$ — PyTorch backend for model inference
- `langchain` $\geq 0.2.0$ + `langgraph` $\geq 0.1.0$ — Agentic pipeline orchestration
- `chromadb` $\geq 0.5.0$ — Persistent vector database for chunk storage and retrieval
- `ragas` $\geq 0.1.10$ — RAG-specific evaluation framework
- `ollama` (CLI) — Local LLM serving for Llama-3-8B-Instruct

4.1.2 Backend and API

- `FastAPI` $\geq 0.111.0$ — Asynchronous REST API server
- `Uvicorn` $\geq 0.29.0$ — ASGI server for FastAPI
- `PyMuPDF(fitz)` $\geq 1.24.0$ — PDF text extraction
- `nltk` $\geq 3.8.1$ — Sentence tokenisation for the Verification Agent
- `pydantic` $\geq 2.0.0$ — Data validation for API request/response models

4.1.3 Frontend Tools

- `Node.js` $\geq 20.x$ LTS + `npm` $\geq 10.x$ — JavaScript runtime and package manager
- `React.js` $\geq 18.x$ — Frontend library for the citation highlighting UI
- `Tailwind CSS` $\geq 3.4.x$ — Utility-first CSS framework
- `Axios` $\geq 1.6.x$ — HTTP client for frontend–backend communication

4.1.4 Development Environment

- Operating System: Ubuntu 22.04 LTS / Windows 11 with WSL2
- Programming Language: Python 3.10+
- IDE: Visual Studio Code with Python and REST Client extensions
- Version Control: Git + GitHub

4.2 Hardware Specifications

4.2.1 Minimum Requirements

- Processor: Intel Core i5 (10th Gen) / AMD Ryzen 5 5000 series
- RAM: 16 GB (required for quantized Llama-3-8B via Ollama)
- Storage: 256 GB SSD (model weights ≈ 4.5 GB + corpus)
- GPU: NVIDIA GPU ≥ 6 GB VRAM (e.g. RTX 3060); CPU-only supported at lower throughput

4.2.2 Recommended Requirements

- Processor: Intel Core i7 (12th Gen+) / AMD Ryzen 7 5800X
- RAM: 32 GB
- Storage: 512 GB NVMe SSD
- GPU: NVIDIA RTX 3080 / RTX 4070 (≥ 10 GB VRAM)

Chapter 5

Project Planning

The project is planned over a fifteen-week period. Tasks are structured to allow iterative development, with early weeks devoted to research and infrastructure setup, mid-project weeks to core agent implementation, and final weeks to evaluation, UI development, and documentation. Documentation runs as a continuous parallel activity throughout all phases. The Gantt chart in Figure 5.1 illustrates the schedule.

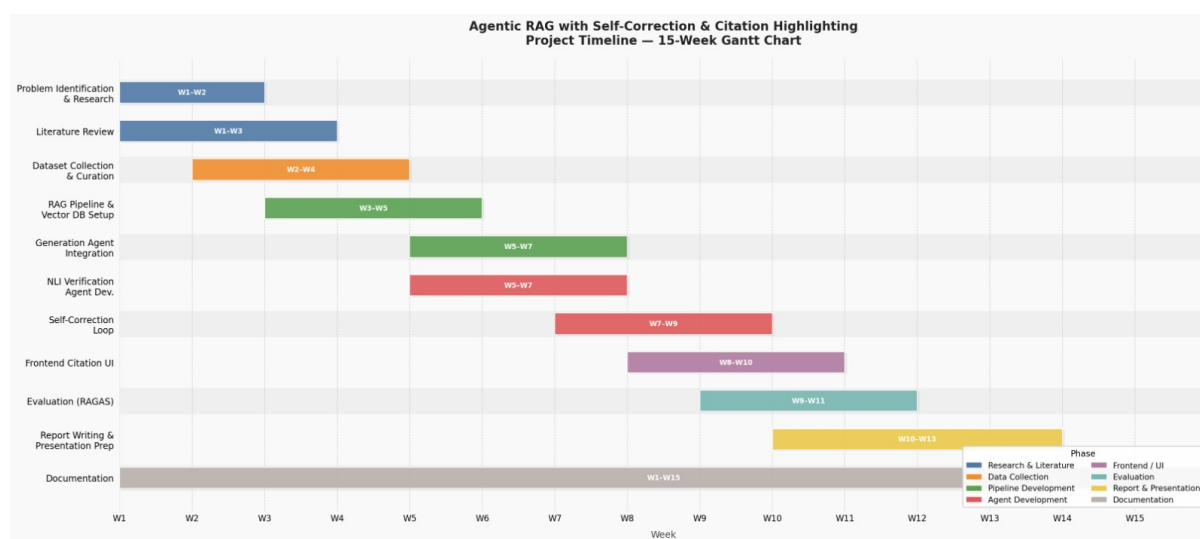


Figure 5.1: Project Timeline (15-Week Gantt Chart)

The key milestones are:

- **Week 1–3:** Problem identification, literature review, and dataset curation. Deliverable: annotated bibliography.
- **Week 3–5:** Document ingestion pipeline and ChromaDB index operational. First end-to-end naive RAG baseline with RAGAS scores.
- **Week 5–7:** NLI Verification Agent integrated. Sentence-level entailment scoring functional.
- **Week 7–9:** Self-Correction Loop implemented in LangGraph. Correction Delta computed against baseline.
- **Week 8–10:** React.js frontend with citation highlighting UI and FastAPI integration.
- **Week 10–13:** Final evaluation, report writing, and presentation preparation.

- **Week 1–15:** Documentation runs continuously throughout all phases as a parallel activity.

Chapter 6

Expected Outcomes

Upon successful completion of this project, the following concrete deliverables and measurable outcomes are anticipated. Quantitative targets are provided in Table 6.1.

- A fully functional Agentic RAG pipeline demonstrably reducing the hallucination rate compared to a standard single-pass RAG baseline, achieving a Faithfulness score greater than 0.90 on the RAGAS evaluation framework.
- A FastAPI-powered backend with a self-correction loop that iteratively revises flagged sentences, achieving a Correction Delta greater than 0.15 relative to the unverified draft.
- A React.js frontend with interactive sentence-level citation highlighting, enabling users to trace every AI-generated claim to its specific source chunk and page.
- A synthetic evaluation dataset of [Context, Question, Hallucinated Answer] triplets generated using a frontier model, serving as a reproducible benchmark for future hallucination detection research.
- A modular, well-documented codebase with reusable prompt templates for the Generation and Correction agents, publishable as an open-source reference implementation.

Table 6.1: Evaluation Metrics and Targets

Metric	Definition	Target
Faithfulness	Fraction of generated sentences entailed by source context	> 0.90
Answer Relevance	Semantic similarity of final answer to user query	> 0.85
Context Recall	Coverage of ground-truth facts by retrieved chunks	> 0.80
Correction Delta	Faithfulness gain from initial draft to corrected output	> 0.15
Hallucination Rate	Fraction of sentences flagged by NLI across eval set	< 0.10

Chapter 7

Ethical Considerations

7.1 Data Privacy

The system is designed to operate entirely on locally hosted models and databases. No user queries or document contents are transmitted to third-party servers unless the Groq API is explicitly enabled as a fallback. When the Groq API is used, users will be clearly informed that their queries are processed externally. Document ingestion will require explicit user consent and will not process personally identifiable information (PII) without appropriate data handling measures. The system will include a data retention policy that allows users to delete indexed documents and associated embeddings from the vector database.

7.2 Bias and Fairness

The NLI model (DeBERTa) was trained on English NLI corpora and may exhibit biases toward particular writing styles, sentence structures, or domains. To mitigate this, the system will be evaluated across diverse document types (scientific, legal, and narrative) and the entailment threshold will be calibrated per domain where feasible. The Generation Agent (Llama-3) may reflect biases present in its pre-training data; however, by grounding generation strictly in retrieved context, the pipeline reduces the model's ability to express parametric biases in factual claims. The citation mechanism further enables users to independently verify claims against primary sources, supporting critical evaluation of the system's output.

7.3 Transparency and Accountability

A core ethical contribution of this project is the citation highlighting interface, which directly addresses the opacity of standard LLM outputs. By making the system's reasoning transparent and auditable, the interface empowers users to identify errors and reduces the risk of uncritical reliance on AI-generated content. The system will clearly distinguish between verified and unverified sentences, and will explicitly disclaim any claim that could not be grounded after the maximum correction iterations. The project will not make claims of human-level reliability;

all outputs should be treated as AI-assisted, not authoritative.

Chapter 8

Risk Analysis

Table 8.1 identifies the primary technical risks associated with this project, their assessed likelihood and impact, and the corresponding mitigation strategies.

Table 8.1: Risk Register

Risk	Likelihood	Impact	Mitigation
LLM fails to follow context-only instruction	Medium	High	Strengthen system prompt; use RAG-tuned models
NLI threshold too strict / lenient	Medium	Medium	Empirically calibrate on held-out validation set
Infinite correction loop	Low	High	Enforce <code>max_iter=3</code> ; omit unverifiable sentences
Insufficient GPU for local inference	Medium	Medium	Use Groq API as fallback for LLM inference
Dataset bias in synthetic eval set	Medium	Medium	Use diverse domains; cross-validate with human eval
Frontend latency due to streaming	Low	Low	Implement progressive streaming from FastAPI

The highest-priority risk is the infinite correction loop, which is fully mitigated by the enforced `max_iter=3` constraint. The second-highest risk is NLI threshold miscalibration, which will be addressed through systematic ablation experiments on a held-out validation set before final evaluation. Overall, the modular architecture of the system means that failures in individual components can be isolated and substituted without requiring a full rebuild of the pipeline.

Chapter 9

Conclusion

This proposal presents Agentic RAG with Self-Correction and Citation Highlighting—a novel multi-agent pipeline that addresses the faithfulness problem in Retrieval-Augmented Generation through an integrated verification-and-correction loop. By separating the concerns of generation, verification, and correction into discrete agents orchestrated by LangGraph, the system systematically eliminates hallucinations before they reach the user, without requiring any fine-tuning of the primary language model.

The architecture contributes three key innovations: (1) the use of an independent NLI model (`cross-encoder/nli-deberta-base`) as a real-time, sentence-level fact-checker integrated directly into the generation loop; (2) a programmatic self-correction mechanism that iteratively revises flagged claims against verified context; and (3) a transparent citation interface that surfaces the system’s internal verification state as an interactive, auditable user experience.

The project is planned over ten weeks and will be evaluated using the RAGAS framework, targeting a Faithfulness score above 0.90 and a novel Correction Delta metric above 0.15. The deliverables include a fully functional system, a synthetic evaluation dataset, and an open-source codebase that can serve as a template for future work on trustworthy, document-grounded question answering. By making verification and citation native features of the RAG pipeline, this project takes a meaningful step toward AI systems that are not only capable but also transparent and trustworthy.

References

- [1] P. Lewis, E. Perez, A. Piktus et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint arXiv:2310.11511*, 2023. [Online]. Available: <https://arxiv.org/pdf/2310.11511>
- [3] Y. Hu, W. Xuan et al., “A self-correcting agentic graph RAG for clinical decision support in hepatology,” *Frontiers in Medicine*, vol. 12, p. 1716327, Dec. 2025. doi: 10.3389/fmed.2025.1716327. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12748213/>
- [4] Anonymous, “Concise and sufficient sub-sentence citations for retrieval-augmented generation,” *arXiv preprint arXiv:2509.20859*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.20859>
- [5] S. Es, J. James, L. E. Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [6] T. Falke, L. F. R. Ribeiro, P. A. Utama, I. Dagan, and I. Gurevych, “Ranking generated summaries by correctness: An interesting but challenging application for natural language inference,” in *Proc. ACL 2019*, pp. 2214–2220.
- [7] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing,” *arXiv preprint arXiv:2111.09543*, 2021.
- [8] Z. Ji, N. Lee, R. Frieske et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [9] V. Karpukhin, B. Oguz, S. Min et al., “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP 2020*, pp. 6769–6781.
- [10] N. F. Liu, K. Lin, J. Hewitt et al., “Lost in the middle: How language models use long contexts,” *arXiv preprint arXiv:2307.03172*, 2023.

- [11] S. Min, K. Krishna, X. Lyu et al., “FactScoring: Fine-grained atomic evaluation of factual precision in long form text generation,” *arXiv preprint arXiv:2305.14251*, 2023.
- [12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP 2019*, pp. 3982–3992.
- [13] W. Shi, S. Min, M. Yasunaga et al., “REPLUG: Retrieval-augmented language model pre-training,” *arXiv preprint arXiv:2301.12652*, 2023.
- [14] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, “Retrieval augmentation reduces hallucination in conversation,” in *Findings of EMNLP 2021*, pp. 3784–3803.
- [15] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” *arXiv preprint arXiv:2212.10509*, 2022.
- [16] L. Wang, C. Ma, X. Feng et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, 2024.
- [17] Q. Wu, G. Bansal, J. Zhang et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [18] S. Yao, J. Zhao, D. Yu et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR 2023*.
- [19] H. Chase, “LangChain,” GitHub, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>